

Angular Components

the building block of every Angular app



Dinesh Rawat

Instructor · Shorter Loop × Snapied

Yesterday we introduced Angular and installed it with the CLI. Today we learn the most important idea in Angular: the **component**. First we understand **why** components exist and why everyone uses them, then we build one and see its parts.

1 What is a component (the simple idea)?

In modern frameworks like Angular, React, and Vue, a **component** is a reusable, self-contained piece of the user interface. It bundles its own **HTML** (the template), its own **CSS** (the styles), and its own **TypeScript** (the logic), all together in one place.

A website is built by assembling many of these pieces. For example:

- Header component
- Navigation component
- Login form component
- Product card component
- Footer component

Instead of writing the same code again and again, you create a component **once** and reuse it wherever you need it.

An example: the repetition problem

Say you are showing products. Without components, you copy the same block for every product:

THE REPEATED WAY

```
<div class="card">
  <h2>iPhone 16</h2>
  <button>Buy</button>
</div>

<div class="card">
  <h2>Samsung S25</h2>
  <button>Buy</button>
</div>
```

Now imagine a hundred products. That is a hundred copies to write and fix. Instead, you make **one** `ProductCard` component and reuse it, just passing in the different data:

THE COMPONENT WAY

```
<product-card name="iPhone 16"></product-card>
<product-card name="Samsung S25"></product-card>
```

The same component displays different data. One piece of code, reused everywhere, each time with its own values.

2 Why components? (15 advantages)

1. **Code reusability** – write once, use many times.
2. **Easy maintenance** – fix a bug in one place instead of many.
3. **Better organisation** – code is split into smaller, manageable parts.
4. **Improved readability** – smaller files are easier to understand.
5. **Faster development** – reuse existing components instead of starting from scratch.
6. **Consistency** – the same button, card, or form looks and behaves the same everywhere.
7. **Independent development** – different developers can work on different components at the same time.
8. **Easy testing** – components can be tested on their own.
9. **Scalability** – large apps are easier to build and extend.
10. **Encapsulation** – each component manages its own logic, styles, and behaviour without affecting others.
11. **Reduced duplication** – removes repeated HTML, CSS, and JavaScript.
12. **Better performance** – many frameworks update only the components that change, not the whole page.
13. **Simpler debugging** – problems can be isolated to one component.
14. **Flexible composition** – complex pages are built by combining smaller components.
15. **Easier collaboration** – teams develop and review components independently, with fewer conflicts.

3 A real-world way to picture it

Think of a **car**. It is made of independent, reusable parts:

- Engine
- Wheels
- Doors
- Steering
- Seats

Each part is independent and reusable. If you need to replace a wheel, you do not rebuild the entire car. Components work the same way in software. A webpage is built by assembling reusable pieces: a header, a sidebar, a product list, a shopping cart, and a footer.

Note: JavaScript by itself does not have a built-in "component" feature. The component idea is provided by frameworks and libraries such as Angular, React, Vue, Svelte, and Web Components. In this course, we use Angular.

4 What is a component in Angular?

Now that we know why components matter, here is what one actually is inside Angular. In Angular, everything you see on the screen is a component, and they are the basic building blocks of the app.

An Angular component is a class, with a template, and a decorator. Those three things together make one component. Let us look at each part.

- **Template** – defines what the user sees. It holds the HTML, along with Angular's special syntax and data bindings.
- **Class** – holds the code for the view. Like a class in Java or C++, it can have **properties** (the data) and **methods** (the logic). We write it in TypeScript.
- **Decorator** – a special marker, written as `@Component`, that adds information (metadata) to the class. Decorating a class with `@Component` is what turns it into an Angular component.

5 Let the CLI create one for us

Yesterday you learned the CLI. This is where it shines. Instead of making files by hand, you ask the CLI:

TERMINAL

```
ng generate component hello
```

Short form: `ng g c hello`. This creates a folder with the component's files, including the main one: `hello.component.ts`. That `.ts` file is where the three parts live.

6 Building a component, part by part

Step 1: the class

We use the `class` keyword and give it a name. By convention the main component is called `AppComponent`. Inside, we add one property to hold the message we want to show:

THE CLASS

```
export class AppComponent {  
  name: string = "Angular";  
}
```

`name` is a property, its type is `string`, its value is `"Angular"`. The `export` keyword lets other parts of the app use this component.

Step 2: import the decorator

The `@Component` decorator comes from Angular, so we import it first:

THE IMPORT

```
import { Component } from "@angular/core";
```

Step 3: apply the decorator and add metadata

We attach the decorator with the `@` symbol, right above the class, and pass it an object with our settings: a **selector** and a **template**.

THE FULL COMPONENT

```
import { Component } from "@angular/core";  
  
@Component({  
  selector: "app-hello",  
  template: `<h1>Hello {{ name }}</h1>`,  
})  
export class AppComponent {  
  name: string = "Angular";  
}
```

That is a complete Angular component: a class, a template, and a decorator, all in one file. The template sits inside **backticks** (```), which let the HTML span multiple lines.

7 The two key parts explained

The template and data binding

Look at this piece of the template:

```
<h1>Hello {{ name }}</h1>
```

The double curly braces `{{ name }}` are **data binding** (interpolation). Angular takes the `name` property from the class and drops its value into the page. Since `name` is `"Angular"`, the page shows **Hello Angular**. Change the property, and the page changes with it.

The selector

The selector is the custom HTML tag for this component. We set it to `app-hello`. Wherever Angular sees this tag:

```
<app-hello></app-hello>
```

Angular **replaces that tag** with the component's template. So `<app-hello></app-hello>` becomes `<h1>Hello Angular</h1>` at runtime. The selector is how you place your component onto a page, exactly like the `<product-card>` tag from earlier.

8 Run it

TERMINAL

```
ng serve
```

THEN OPEN

```
http://localhost:4200
```

You will see **Hello Angular**. The `app-hello` tag was replaced by the template, and `{{ name }}` was filled in from the class. A component, working end to end.

9 Remember this

A component is a reusable, self-contained piece of UI: its own HTML, CSS, and logic in one place.

Why: reuse, easy maintenance, consistency, testing, teamwork. Write once, use everywhere.

In Angular, a component = class + template + decorator.

Class holds data and logic, **template** is the HTML the user sees (with `{{ }}` binding), **decorator** `@Component` marks the class and adds the selector and template.

Selector: the custom tag (like `app-hello`) that Angular replaces with the template.

The CLI makes one with `ng generate component <name>`.

10 Homework

Generate a component with `ng g c hello` and open its `.ts` file. Find the three parts: class, template, decorator.

Add a property like `city: string = "Haldwani";` and show it in the template with `{{ city }}`.

Think of three things on any website you use that are probably components, and why making them reusable helps.

In one line each: what do the class, the template, and the decorator do?